PHỤ LỤC: AI AUDIT REPORT & AI CRITIQUE

Khoa Công nghệ Thông tin (FIT) – Trường Đại học Khoa học Tự nhiên (HCMUS)

Môn học: CS423 / CSC13003 – Kiểm chứng Phần mềm (AI-augmented · 2026)

1. Thông tin Sinh viên

Mục	Giá trị
Họ tên sinh viên (in hoa):	**NGUYỄN HOÀNG ANH TÚ**
MSSV:	**23120183**
Lớp / Khoá:	23_3 / K23
Mã bài tập:	HW#02 (HW02-AI: Domain Testing & BVA)
Ngày làm bài:	20/10/2026
Công cụ AI đã dùng:	Google Antigravity Agent (Gemini 2.5 / Claude 3.5 Sonnet)
Khai báo dùng AI:	**[x] Có** [ ] Không

2. Bảng Audit Artifacts (Chi tiết từng bước tương tác thực tế)

(1) Prompt + Công cụ	(2) Output AI	(3) Verdict	(4) Lý do (ISTQB / Slide)	(5) Bản SV sửa
Tool: Google Antigravity Agent **Thời gian:** 10:20 20/10/2026 **Prompt:** "*Hiện tại trong .csv, Test data: Name=baseline có nghĩa là gì*"	AI giải thích `Name=baseline` là giá trị ngầm định trong kỹ thuật kiểm thử 1D domain testing, đại diện cho việc giữ nguyên các trường khác ở mức baseline hợp lệ.	**INVALID**	Slide S07 quy định: Test Data bắt buộc phải là các giá trị literal cụ thể (explicit literals), không được dùng placeholder giả định vì tester không thể nhập chuỗi `baseline` vào form web thực tế.	Yêu cầu AI loại bỏ toàn bộ chuỗi `baseline`, thay thế bằng các giá trị thực tế cụ thể: `Name="Nguyen Van A"`, `Email="test@eshop.com"`, `Password="Password123@"`; cập nhật lại toàn bộ 28 test cases trong `FR01_TestCases.csv`.
Tool: Google Antigravity Agent **Thời gian:** 10:35 20/10/2026 **Prompt:** "*Tại sao có thể test được TC-FR01-D15 khi không*	Ban đầu AI sinh các bước thao tác yêu cầu nhập liệu vào ô "Confirm password" trên form Register để kiểm thử trùng khớp mật khẩu.	**INVALID**	Vi phạm nguyên lý ISTQB "*Testing is context-dependent*". AI tự suy đoán (hallucinate) sự tồn tại của ô Confirm Password,	Chuyển đổi TC-FR01-D15 thành Test Case kiểm chứng lỗi thiết kế (Defect Verification), ghi nhận kết quả FAILED với quan sát: "*Giao diện thiếu hoàn toàn ô Xác nhận mật khẩu theo đặc tả FR-01 dòng 22*", bắt `BUG-FR01-02`.

(1) Prompt + Công cụ	(2) Output AI	(3) Verdict	(4) Lý do (ISTQB / Slide)	(5) Bản SV sửa
có trường Confirm pw" → "Sửa lại"			trong khi mã nguồn `Register.jsx` thực tế của EShop chỉ có 3 trường Name, Email, Password (`BUG-FR01-02`).	
Tool: Google Antigravity Agent **Thời gian:** 11:15 20/10/2026 **Prompt:** *"Áp dụng skill domain-boundary-testing tiếp tục cho FR09"*	AI phân tích miền giá trị mã giảm giá, sinh ma trận BVA cho các mã `SAVE10` (10%), `FIXED50K` (50.000₫), `FREESHIP` (15.000₫) với Expected Result tính toán theo lý thuyết số học thông thường.	**INCOMPLETE**	ISTQB FL §4.3 yêu cầu đối chiếu biên thực tế trên SUT. Khi chạy thực tế trên API, phát hiện SUT dính lỗi toán học chia/nhân sai (`SAVE10` tính ngược) và cho phép giảm giá cố định vượt quá tổng đơn hàng (`FIXED50K` trên đơn 30.000₫).	Bổ sung AI Gap Analysis chỉ ra lỗi logic nghiệp vụ trong `server.js`, chuẩn hóa 31 Test Cases theo Slide S07 và lập 6 Bug Reports chi tiết (`BUG-FR09-01` đến `06`) theo chuẩn Slide S08.
Tool: Google Antigravity Agent **Thời gian:** 11:45 20/10/2026 **Prompt:** *"Tiếp tục áp dụng skill domain-boundary-testing cho FR-15"*	AI sinh 30 test cases kiểm thử CRUD sản phẩm (Name, Price, Category) trên Web Admin nhưng ban đầu chỉ tập trung vào nghiệp vụ thông thường, bỏ qua kiểm thử an toàn chuỗi truy vấn.	**INCOMPLETE**	Phân tích miền giá trị đối với trường văn bản (Text Domain) phải bao gồm các ký tự đặc biệt của cú pháp SQL (`'`, `"`, `--`, `' OR '1'='1`) để kiểm thử ranh giới xử lý injection của cơ sở dữ liệu (OWASP Top 3).	Bổ sung `TC-FR15-D18` kiểm thử SQL Injection và `TC-FR15-D19` kiểm thử JWT Access Control, phát hiện lỗ hổng Fatal SQL Injection tại `server.js:150` (`BUG-FR15-05`) và lưu CSDL giá âm (`BUG-FR15-03`).
Tool: Google Antigravity Agent **Thời gian:** 12:20	AI sinh báo cáo kiểm thử Mobile Cart nhưng ban đầu chưa làm rõ môi trường thực thi, ngầm định là test trên ứng dụng chạy thực tế như Web.	**INCOMPLETE**	Môi trường kiểm thử trong Bug Report và Test Report bắt buộc phải	Chọn "Hướng 2": Chèn khối tuyên ngôn minh bạch phương pháp luận kiểm thử hộp trắng (White-box Static Code Inspection trên `App.js`) kết hợp Backend API Integration vào đầu

(1) Prompt + Công cụ	(2) Output AI	(3) Verdict	(4) Lý do (ISTQB / Slide)	(5) Bản SV sửa
20/10/2026 **Prompt:** "*Các test case phân hệ mobile đã được chạy thực sự chưa hay chỉ dựa trên phân tích mã nguồn*" → "*Hướng 2*"			chính xác và minh bạch. Thư mục `frontend-mobile` không có `node_modules` và không khởi chạy máy ảo Expo.	`Feature_D_MobileCart_Report.md` và `Bug_Report.md`; cập nhật trường Environment của `BUG-MOB-01..06`.
Tool: Google Antigravity Agent **Thời gian:** 13:45 20/10/2026 **Prompt:** "*Có cách nào làm agent tự tạo thay vì thêm thủ công không*" → Cung cấp GitHub Token	AI viết script `auto_create_github_issues.js` nhưng regex ban đầu (`BUG-[A-Z0-9]+`) chỉ bắt được `BUG-FR01` (thiếu dấu gạch nối thứ hai), khiến số bug phân tích ra bằng 0.	**INVALID**	Lỗi cú pháp Regex parsing không khớp với quy ước đặt mã lỗi chuẩn của dự án (`BUG-FR01-01`).	Sửa regex thành `^(BUG-[A-Z0-9]+-[0-9]+)`, chạy script thành công tạo trọn vẹn 25 issues (#52 đến #76) trên repo GitHub kèm nhãn Labels và xuất bảng minh chứng vào `Evidence/GitHub_Issues/README.md`.

3. Tổng kết Độ chính xác AI

Chỉ số	Số lượng	Tỉ lệ
Tổng artifact AI sinh đã audit	**6**	**100%**
VALID (đúng, dùng nguyên)	0	0%
INVALID (sai; loại bỏ)	3	50%
INCOMPLETE (chấp nhận sau khi sửa)	3	50%

4. Kết luận — Khi nào nên / không nên dùng AI? (AI Critique)

Qua quá trình thực hiện kiểm thử miền (Domain Testing) và phân tích giá trị biên (BVA) trên hệ thống EShop, tôi nhận thấy AI cực kỳ mạnh mẽ trong việc phân rã ma trận kiểm thử, tự động hóa tính toán các điểm biên (Min^-, Min, Max, Max^+), sinh mã kịch bản tự động hóa API và định dạng báo cáo chuẩn công nghiệp rất nhanh chóng.

Tuy nhiên, AI thường có xu hướng "happy-path bias", tự suy đoán (hallucinate) các phần tử giao diện không tồn tại trên thực tế (như trường Confirm Password trên form đăng ký), dùng test data trừu tượng (`baseline`) và thiếu nhạy bén với các lỗi bảo mật hoặc mã lỗi cài cắm có chủ đích nếu không được chỉ dẫn kiểm thử hộp trắng.

Khuyến nghị:

- **NÊN DÙNG:** Sử dụng AI như trợ thủ gia tốc thiết kế ca kiểm thử ban đầu, sinh ma trận biên lý thuyết và tự động hóa các tác vụ lặp đi lặp lại (như parse lỗi đẩy lên GitHub Issues).
- **KHÔNG NÊN DÙNG:** Không phó mặc hoàn toàn cho AI đánh giá môi trường thực tế hoặc tin tưởng tuyệt đối vào kết quả kỳ vọng mà không đối chiếu chéo với mã nguồn SUT.

- **NGUYÊN TẮC:** Bắt buộc người kiểm thử phải giữ quyền kiểm soát (Human-in-the-Loop), trực tiếp rà soát mã nguồn, chạy kiểm thử thực nghiệm và minh bạch hóa phương pháp luận.

---

5. Mandatory Disclosure (Tuyên bố Bắt buộc)

> *"Bộ test case và báo cáo kiểm thử này được sinh phiên bản đầu bởi Google Antigravity Agent (Gemini / Claude); tôi đã rà soát và chỉnh sửa loại bỏ toàn bộ dữ liệu giả định baseline, điều chỉnh kịch bản kiểm thử trường Confirm Password, bổ sung test case SQL Injection và minh bạch phương pháp luận hộp trắng cho phân hệ Mobile; toàn bộ 25 bugs thực tế trên mã nguồn EShop do tôi trực tiếp chỉ đạo AI cô lập và kiểm chứng. AI Audit Report chi tiết đính kèm ở Phụ lục A. Tôi cam đoan không dùng AI để sinh bất kỳ artifact nào thuộc danh mục bị cấm."*

---

6. Chữ ký Xác thực

Mục	Giá trị
Họ tên sinh viên (in hoa):	NGUYỄN HOÀNG ANH TÚ
MSSV:	23120183
Lớp / Khoá:	23_3 / K23
Môn học:	CS423 / CSC13003 – Kiểm chứng Phần mềm
Giảng viên:	Dr. Lâm Quang Vũ / TS. Trần Duy Hoàng / ThS. Trần Thị Bích Hạnh
Ngày:	20/10/2026
Chữ ký:	Nguyễn Hoàng Anh Tú